[CDT204 – LECTURE 02]

Instruction Set Architectures (ISA)

Afshin Ameri

Computer Architecture · Mälardalen University · Autumn 2026

Outline

01 • Instruction Set Architectures

02 • Binary representations

03 • ISA and Binary Representation



01

Instruction Set Architecture

Instruction Set Architectures (ISAs)

- Hardware/Software interface
- Specifies:
  - Available instructions
  - Addressing modes
  - Registers
  - …
- MIPS, ARM, x86, m68k, …

Layers of computer architecture

1. ISA
2. Computer organization / microarchitecture
3. Hardware

Classes of ISAs

- Stack
- Accumulator
- Register-memory
- Register-register (or load/store)

CISC and RISC

- CISC: Complex Instruction Set Computer
  - “Source code-like” instructions.
  - Complex hardware
- RISC: Reduced Instruction Set Computer
  - Simpler instructions
  - Simpler hardware → more efficient
  - Might be harder to write the code by hand

Instructions

Example C code

```
int vals[5]={3,-4,-9,10,2};
int sum_pos=0;
for(int i=0;i<5;++i)
   if(vals[i]>0)
      sum_pos+=vals[i];
Printf(“sum_pos:%d\n”,sum_pos);
```

Margin note

Hardware must support:

- Arithmetic operations
- Loading from memory
- Storing to memory
- Comparisons
- Deciding program flow
- Function calls
- …

The MIPS ISA

- Register-register architecture
  - 32 bit or 64 bit word sizes
- Invented in 1981 at Stanford University
- Design goal: simple and fast
- Used mostly in embedded systems

Arithmetic operations in MIPS

- Format

  ```
  op a, b, c
  ```

- Example

  ```
  add $s0, $s1, $s2
  ```

- *Design principle 1*

  Simplicity favors regularity

Example

- C code

```
f = (g + h) – (i + j);
```

- Compiled MIPS code

```
add t0,g,h          #temp t0=g+h
add t1,i,j          #temp t1=i+j
sub f,t0,t1         #f=t0-t1
```

Registers in MIPS

- MIPS has a 32x32-bit register file
  - Numbered 0 to 31
  - 32-bit data is called a word
- Assembler names
  - `$t0, $t1, …, $t9` for temporary values
  - `$s0, $s1, …, $s7` for saved variables
  - …
- *Design principle 2*
  Smaller is faster

Example: register operands

- C code

```
f=(g+h)-(i+j)
```

- Compiled MIPS code

```
add $t0,$s1,$s2
add $t1,$s3,$s4
sub $s0,$t0,$t1
```

Var.	Reg.
F	$s0
G	$s1
H	$s2
I	$s3
J	$s4

Registers of MIPS

Name	Reg. Number	Usage	Preserved on call
`$zero`	0	Constant value 0	N.A.
`$at`	1	Reserved for assembler	N.A.
`$v0-$v1`	2-3	Values for results and expressions	No
`$a0-$a3`	4-7	Arguments (procedures/functions)	Yes
`$t0-$t7`	8-15	Temporaries	No
`$s0-$s7`	16-23	Saved	Yes
`$t8-$t9`	24-25	Extra temporaries	No
`$k0-$k1`	26-27	Reserved for the Operating System	N.A.
`$gp`	28	Global pointer	Yes
`$sp`	29	Stack pointer	Yes
`$fp`	30	Frame pointer	Yes
`$ra`	31	Return address	Yes

Memory operands in MIPS

- Memory is used for composite data
  - Arrays, structures, dynamic data
- Addressed byte-wise
  - Each address identifies an 8-bit value
- Loads/stores must be aligned
  - Address multiple of size of read/write

Exmaple: Memory operands

- C code

```
A[9]=A[8]+g;
```

- Compiled MIPS code (`g` in `$s0`, base address of `A` in `$s1`)

```
lw  $t0, 32($s1)     #load word
add $s2,$t0,$s0      #compute
sw  $s2, 36($s1)     #store result
```

Registers vs. Main Memory

- Registers are way faster than memory
- Operating on memory data requires load and stores
  - More instructions
- Compiler should use registers for variables as much as possible
  - Only spill to memory for less frequently used variables

Immediate operands in MIPS

- Constant operands

```
addi $s3, $s3, 4
```

- No “subtract immediate” function

```
addi $s2, $s1, -1
```

- *Design principle 3*
  Make the common case fast
- *Design principle 4*
  Good design demands good compromises

The `$zero` register

- Always holds the value 0
  - Cannot be overwritten
- Useful for common operations
  - `add  $t0,$s1,$zero`
  - `sub  $t3,$zero,$s5`
  - `addi $t6,$zero,19`

Making decisions (Branching)

- `beq rs, rt, L1`
  - If `(rs==rt)` branch to instruction labeled `L1`
- `bne rs, rt, L1`
  - if `(rs!=rt)` branch to instruction labeled `L1`
- `j L1`
  - Unconditional jump to instruction labeled `L1`
- Labels are translated by assembler

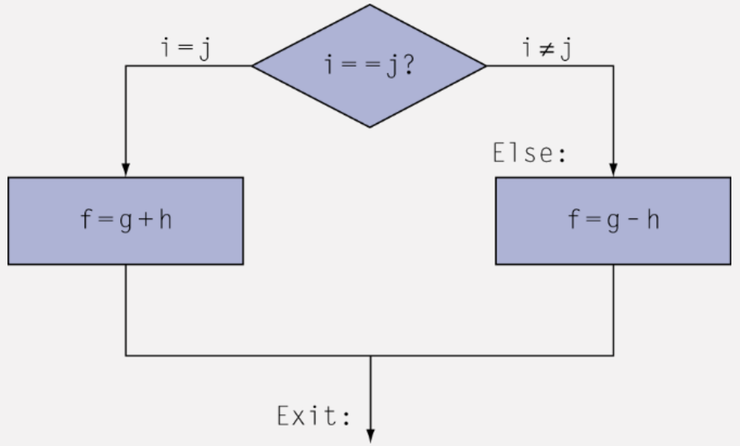
Example: Branching

- C code

```
if(i==j) f=g+h;
else f=g-h;
```

- Compiled MIPS code

```
     bne $s3,$s4, Else
     add $s0,$s1,$s2
     j   Exit
Else:sub $s0,$s1,$s2
Exit:...
```



Var.	Reg.
F	$s0
G	$s1
H	$s2
I	$s3
J	$s4

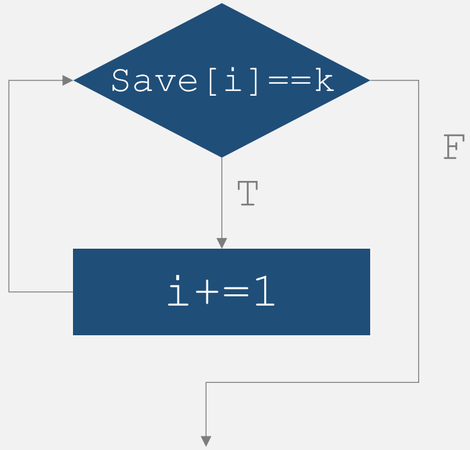
Example: Loops

- C code

```
while(save[i]==k)
     i+=1;
```

- Compiled MIPS code

```
Loop:sll  $t1,$s3,2
     add  $t1,$t1,$s6
     lw   $t0,0($t1)
     bne  $t0,$s5,Exit
     addi $s3,$s3,1
     j    Loop
Exit:...
```



Var.	Reg.
I	$s3
K	$s5
Save	$s6

Branching

- Set result to 1 if a condition is true otherwise set to 0

```
slt  rd,rs,rt          #if(rs<rt) rd=1; else rd=0;
slti rt,rs,const       #if(rs<const)rt=1;else rt=0;
```

- Use in combination with beq, bne

```
slt $t0,$s1,$s2     #if($s1<$s2)
bne $t0,$zero,L     #branch to L
```

Procedure call

- Procedure call: Jump and link

  ```
  jal ProcedureLabel
  ```

  - Address of the following instruction put in $ra
  - Jumps to target address
- Procedure return: Jump register

  ```
  jr $ra
  ```

  - Copies $ra to program counter
  - Can also be used for computed jumps
    - e.g. for switch/case statements

Pseudoinstructions

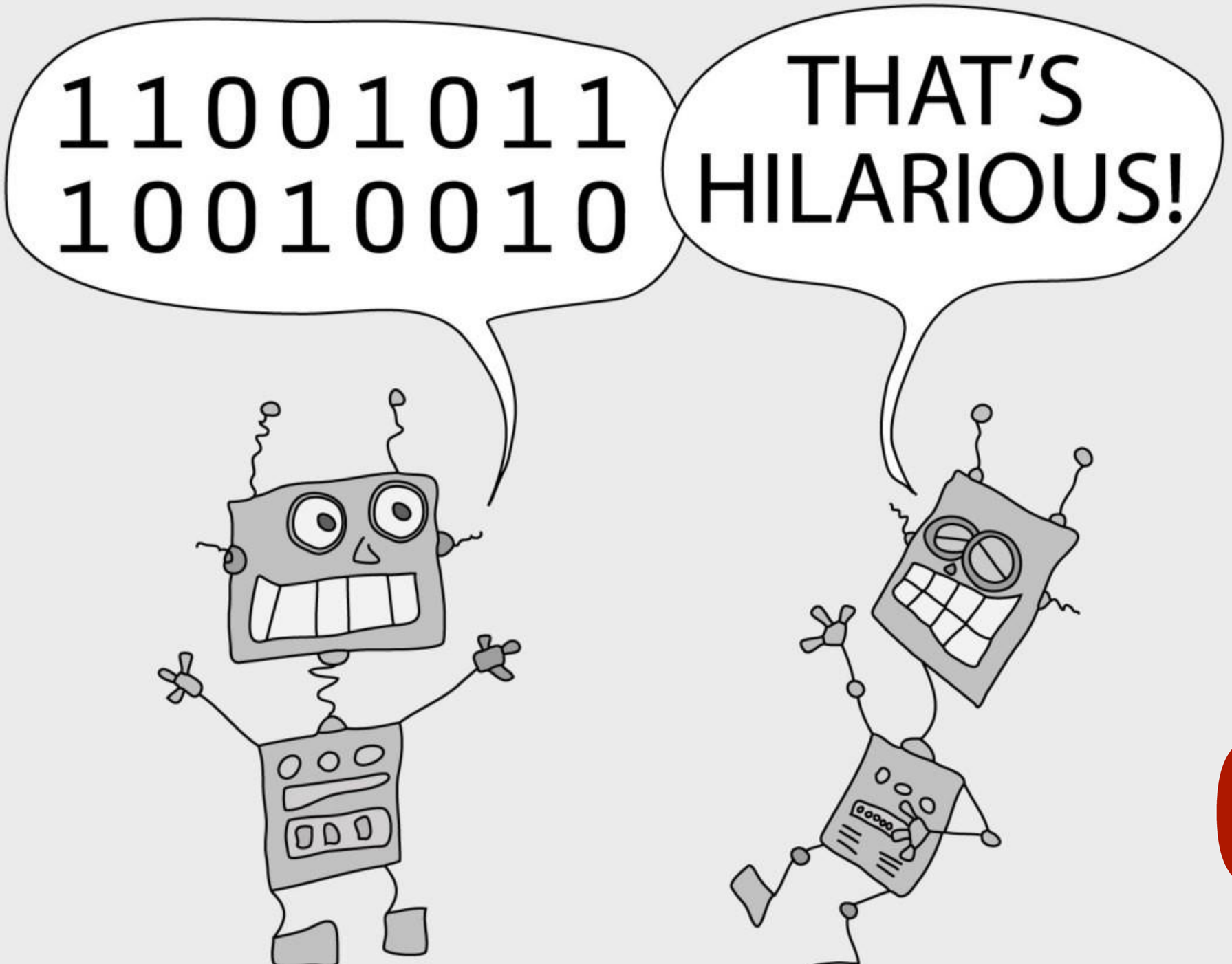
- Not implemented in hardware
  - Simplifies the assembler code
  - Translated to real instructions by assembler
- Examples:
  - `move`: copy values between registers
  - `clear`: store 0 in register
  - `blt`: branch if less than

Other ISAs: ARM

- RISC
- Most widespread ISA for embedded systems
- Many aspects in common with MIPS
  - Fixed length encoding (32 bits)

Other ISAs: x86

- A.K.A. IA-32
- CISC
  - Hundreds of instructions
  - Ugly ☹
- First implementation in 1978
- Most widespread ISA for desktops



02 Binary Representations

Binary numbers

COUNTING IN BASE 10

0
1
2
...
9
10
11
...
99
100
101
...

COUNTING IN BASE 2

0
1
10
11
100
101
110
111
1000
1001
1010
...

Unsigned binary numbers

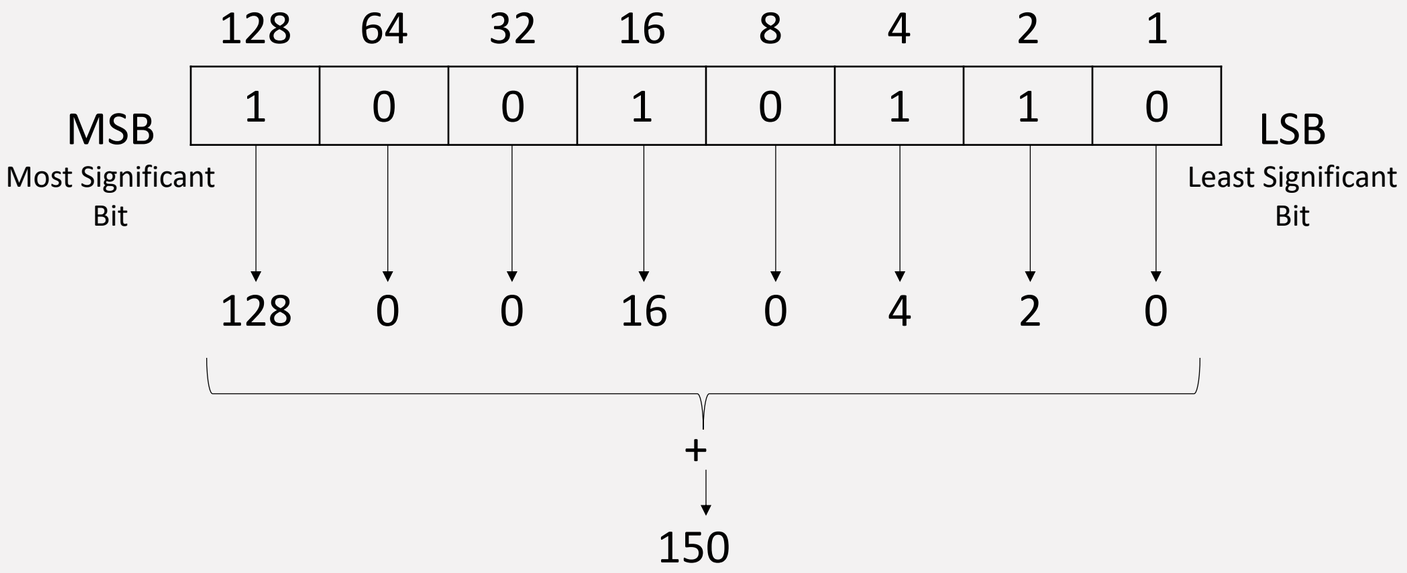
- A binary number $x_{n-1}x_{n-2}\cdots x_1x_0$ represents:

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2}+\ldots+x_1 2^1+x_0 2^0$$

- Range [0, 2^n-1]: n=32 → [0, 4,294,967,295]
- Example:

$$00001011_2 = 0\times 2^7 + \cdots + 1\times 2^3 + 0\times 2^2 + 1\times 2^1 + 1\times 2^0$$
$$= 0 + \cdots + 8 + 0 + 2 + 1 = 11_{10}$$

Unsigned binary



Hexadecimal representations

COUNTING IN BASE 10

```
0,
 1,  2, …,  9, 10,
11, 12, …, 19, 20,
…
91, 92, …, 99, 100,
…
… 999, 1000
```

COUNTING IN BASE 16

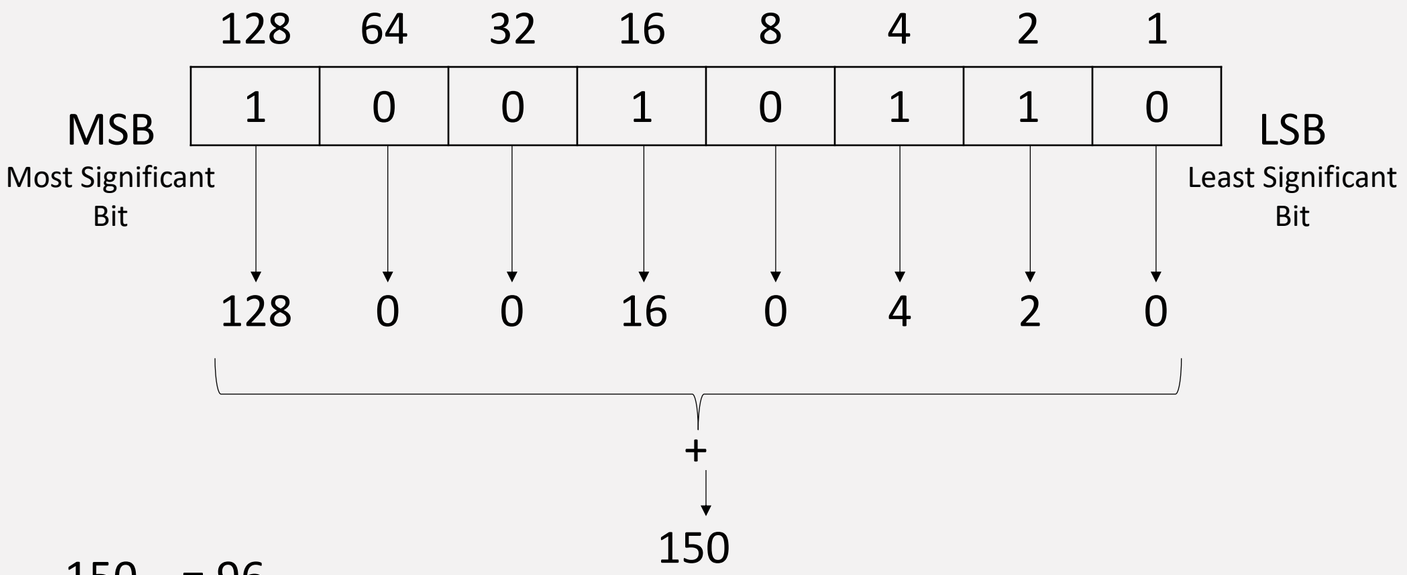
```
 0,
 1,  2, …,  9,  A,  B,  C,  D,  E,  F, 10,
11, 12, …, 19, 1A, 1B, 1C, 1D, 1E, 1F, 20,
…
F1, F2, …, F9,…, FF, 100,
…
…, FFF, 1000
```

Hexadecimal representation

- In binary groups of 4bits can represent 2^4=16 different values
- Using 16 symbols (0-9 plus A-F), such a 4-group can be represented with a single symbol:

  $1101000111110010_2 = D1F2_{16}$
- C-like languages use the prefix “0x” for hexadecimal values

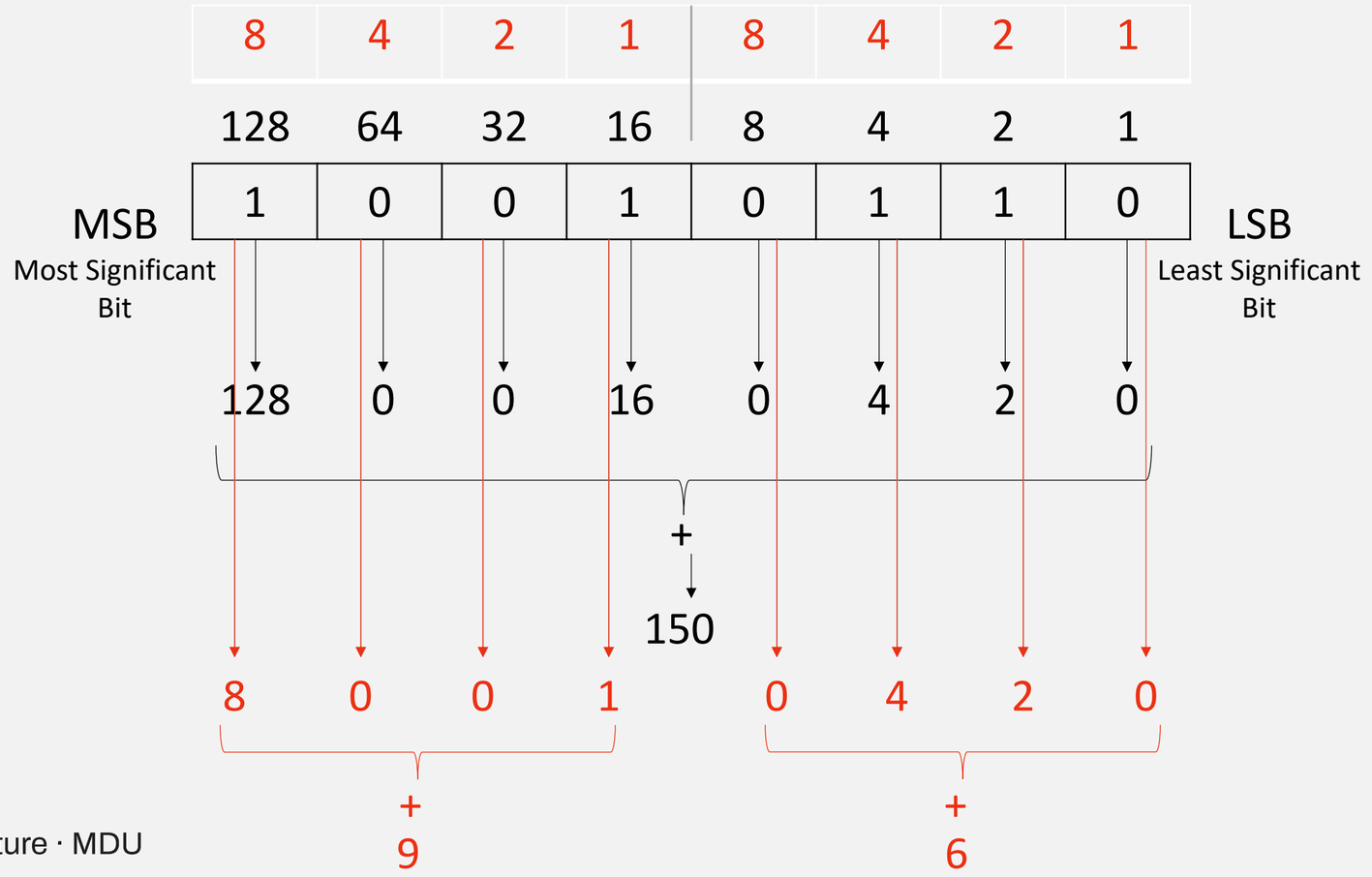
Unsigned binary



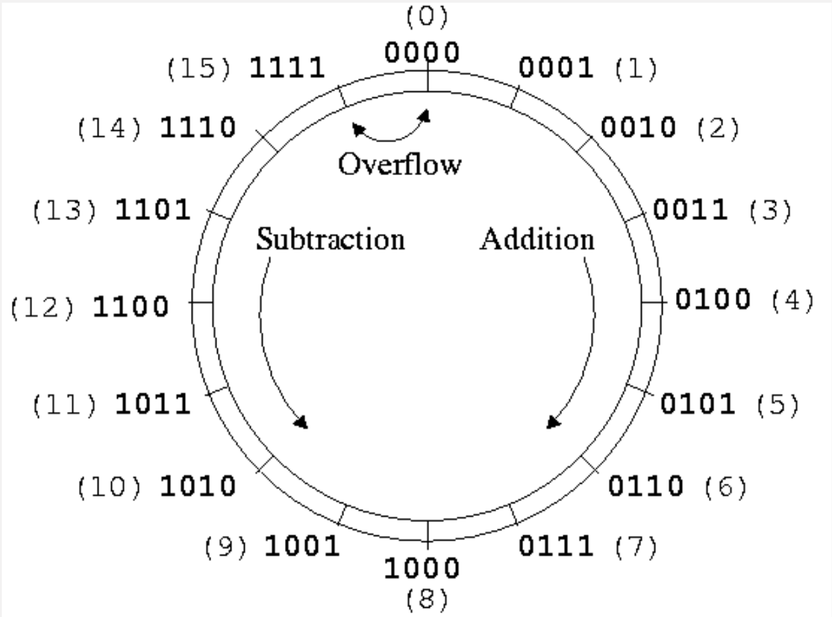
$150_{10} = 96_{16}$

$$96_{16} = 9 \times 16^1 + 6 \times 16^0 = 144 + 6 = 150_{10}$$

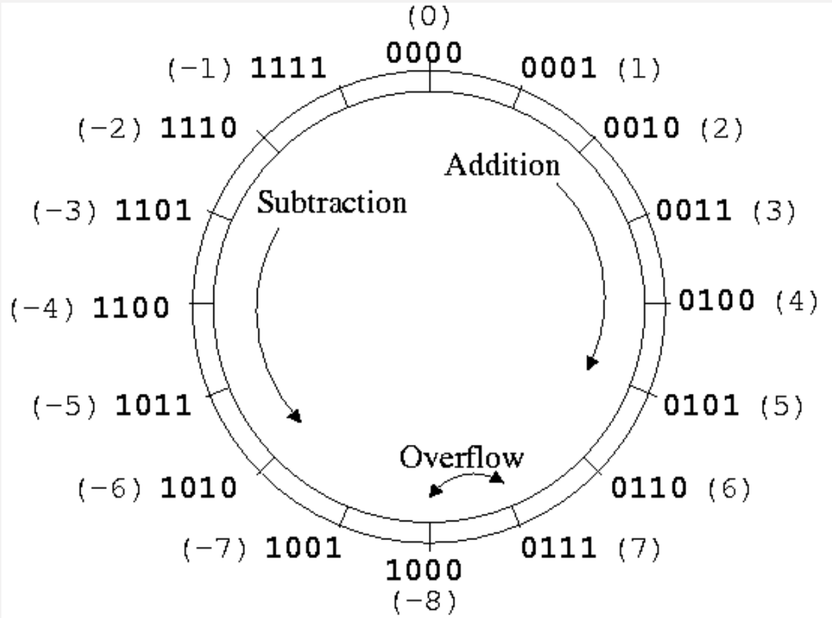
Unsigned binary: Hexadecimal



Complement to base



unsigned



signed

Signed integers

- What about the signed integer values?
- Two’s complement is the most widely used technique
- It is easy to calculate and makes chip-design easier.

Signed binary numbers

- A binary number $x_{n-1}x_{n-2}\cdots x_1x_0$ represents:

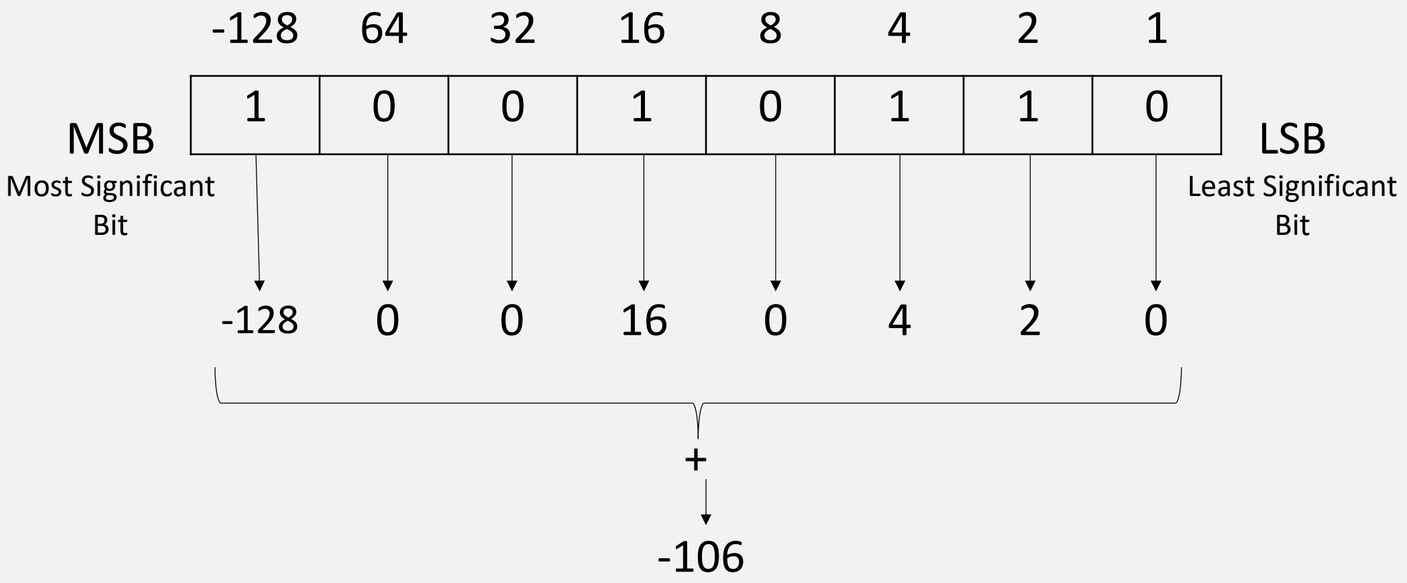
$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2}+\ldots+x_1 2^1+x_0 2^0$$

- Range [- 2^{n-1}, 2^{n-1}-1]:

  – n=32 → [-2147483648, -2147483647]
- Example:

$$11111100_2 = -1 \times 2^7 + 1 \times 2^6 + \cdots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$
$$= -128 + 64 + 32 + 16 + 8 + 4 = -4_{10}$$

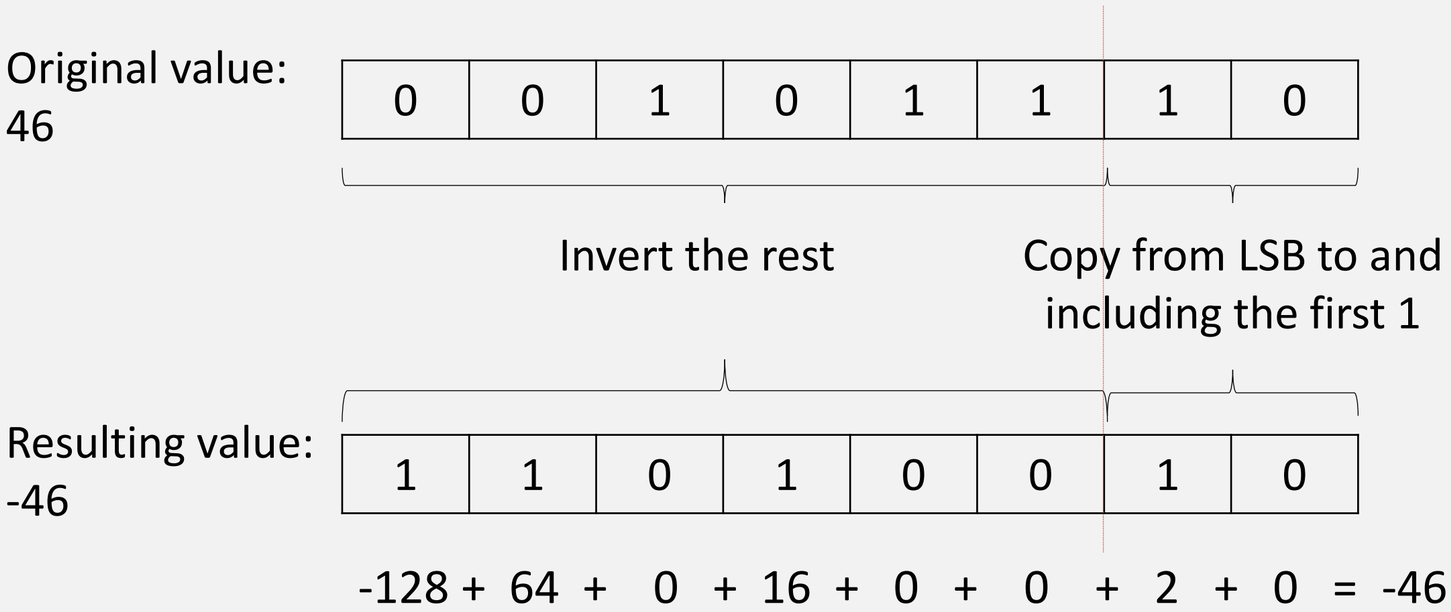
Two’s complement



Two’s complement

- Given a number *m*, what is the binary representation of –*m* in two’s complement?
- Invert all the bits. Add 1 to the result.
- Or
    1. Copy from LSB to and including the first 1
    2. Invert all remaining bits

Example: Two’s complement



Two’s complement

- Leftmost bit tells the sign
- Non-negative numbers have the same unsigned and two’s complement representation
- Some specific numbers
  - 0 0000 0000 0000 0000
  - -1 1111 1111 1111 1111
  - Smallest negative 1000 0000 0000 0000
  - Largest positive 0111 1111 1111 1111

Exercise

- Translate the decimal number *81* to an 8-bit two’s complement binary number.
- Translate the decimal number *-94* to an 8-bit two’s complement binary number.
- Translate the unsigned binary number *1110111100111010* to a hexadecimal number.

Zero/sign extension

Going from an *n*-bit representation to an *m*-bit representation, where *m>n*

- For unsigned values: zero extend
  0101 → 00000101
  1101 → 00001101
- For signed values: sign extend
  0101 → 00000101
  1101 → 11111101

Bitwise operations

AND

1	1	1	0	0	1	0	1
1	0	0	1	1	1	0	1
1	0	0	0	0	1	0	1

In C: a & b

OR

1	1	1	0	0	1	0	1
1	0	0	1	1	1	0	1
1	1	1	1	1	1	0	1

In C: a | b

Bitwise operations

NOT

1	1	1	0	0	1	0	1
0	0	0	1	1	0	1	0

In C: ~ a

XOR

1	1	1	0	0	1	0	1
1	0	0	1	1	1	0	1
0	1	1	1	1	0	0	0

In C: a ^ b

Bitwise operations

Left shift: $11100101_2 << 3_{10} = 00101000_2$

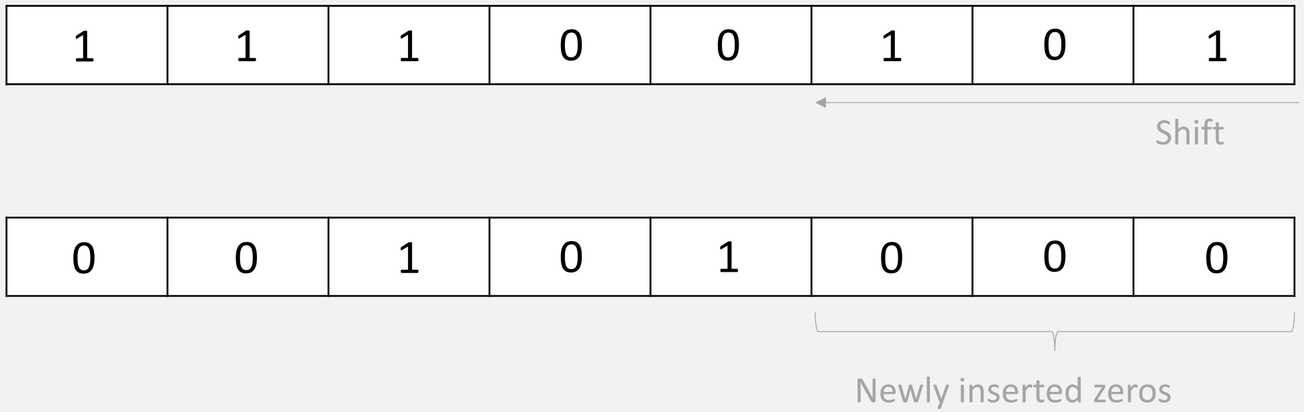
1	1	1	0	0	1	0	1

✖ ✖ ✖

1	1	1	0	0	0	0	0

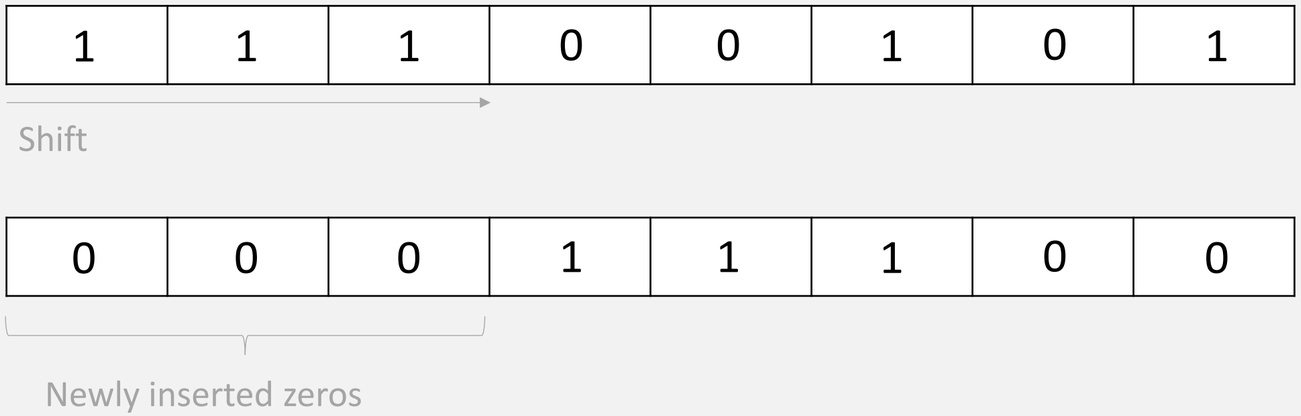
Bitwise operations

Left shift: $11100101_2 << 3_{10} = 00101000_2$



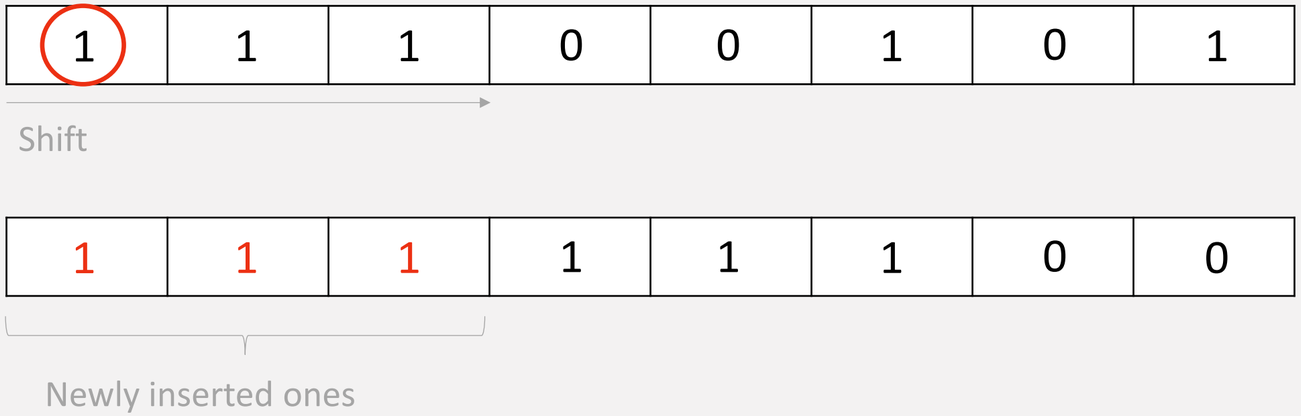
Bitwise operations

Logical right shift: $11100101_2 >> 3_{10} = 00011100_2$



Bitwise operations

Arithmetic right shift: $11100101_2 >> 3_{10} = 11111100_2$



Example: Bitwise operations

```
int abs(int x)
{
     int sign=x>>31;
     x=(x^sign)-sign;
     return x;
}
```

Margin note

- If x<0
  sign=111…111_2= -1_{10}
  x= ~x-(-1)

- If x>0
  sign=000…000_2=0_{10}
  x=x-0

Example: Bitwise operations

```
int sign=x>>31;
x=(x^sign)-sign;
```

This code is written for a 32-bit value. In this example we use 7 instead of 31 for an 8-bit value.

❶ x=-53

1	1	0	0	1	0	1	1

❷ sign=x>>7 ⟶ sign=-1

1	1	1	1	1	1	1	1

```
x=(x^(-1))-(-1) →
  x=(x^(-1))+1
```

❸

x	1	1	0	0	1	0	1	1
	XOR							
-1	1	1	1	1	1	1	1	1
x^(-1)	0	0	1	1	0	1	0	0

❹

	+							
1	0	0	0	0	0	0	0	1
x^(-1)+1	0	0	1	1	0	1	0	1

=53

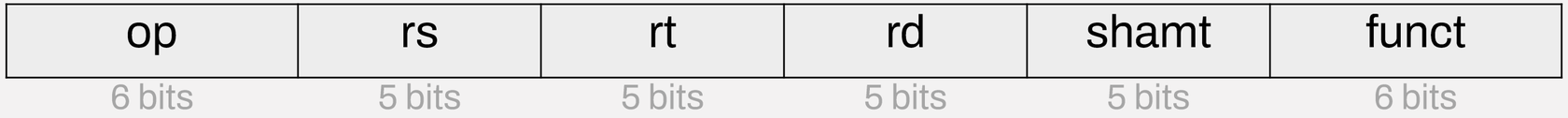
03

ISA and Binary Representation

Representing MIPS Instructions

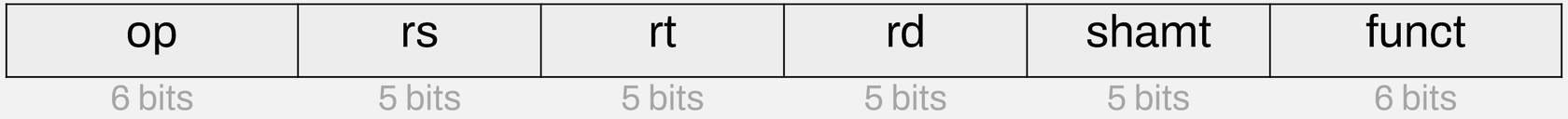
- Encoded as 32-bit instruction words
- Small number of formats, encoding the following
  - Operation code (opcode)
  - Register numbers
  - ...
- Regularity

MIPS R-Format Instructions



- Instruction fields
  - **op**: operation code (opcode)
  - **rs**: first source register number
  - **rt**: second source register number
  - **rd**: destination register number
  - **shamt**: shift amount (00000 for now)
  - **funct**: function code (extends opcode)
- Used for arithmetic instructions

Example: R-Format

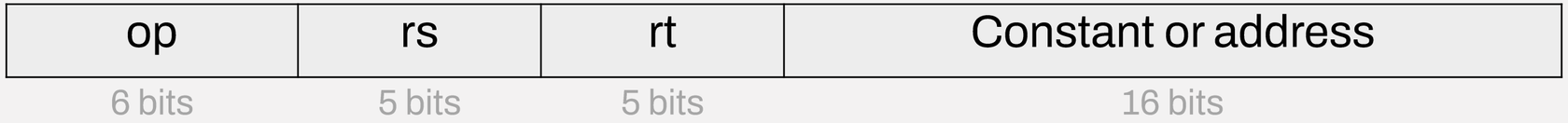



```
add $t0, $s1, $s2
```

Fields:	special	$s1	$s2	$t0	0	add
Decimal:	0	17	18	8	0	32
Binary:	000000	10001	10010	01000	00000	100000

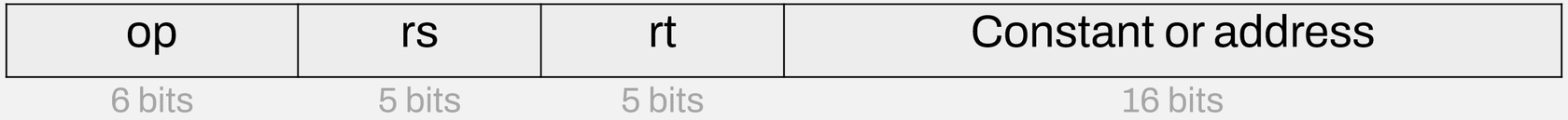
$$00000010001100100100000000100000_2 = 02324020_{16}$$

MIPS I-Format Instructions



- Instruction fields
  - **op**: operation code (opcode)
  - **rs**: source register number
  - **rt**: source/destination register number
  - **Constant or address**: signed 16-bit value
- Used for branch, transfer and immediate instructions

Example: I-Format




```
lw $t0, 1200($t1)
```

Fields:	lw	$t1	$t0	1200
Decimal:	35	9	8	1200
Binary:	100011	01001	01000	0000010010110000

$$10001101001010000000010010110000_2 = 8D2804B0_{16}$$

MIPS J-Format Instructions



- Instruction fields
  - **op**: operation code (opcode)
  - **target address**: 26 bit address
- Used for jump instructions

Example: I-Format




```
j 10000
```

Fields:	j	10000
Decimal:	2	10000
Binary:	000010	00000000000010011100010000

$$000010\ 00000000000010011100010000_2 = 8002710_{16}$$

From source to machine code

```
A[300]=h+A[300];
```

Compiler

```
lw  $t0, 1200($t1)
add $t0, $s2, $t0
sw  $t0, 1200($t1)
```

Assembler

35	9	8	1200		
0	18	8	8	0	32
43	9	8	1200		

`$t1` ← base of array A
`$s2` ← h

Temporary register `$t0` gets `A[300]`

Temporary register `$t0` gets `h+A[300]`

Stores `h+A[300]` back into `A[300]`

100011	01001	01000	0000 0100 1011 0000		
00000	10010	01000	01000	00000	10000
101011	01001	01000	0000 0100 1011 0000		

From source to machine code

Address	Code and Format		Basic	Source
0x00400000	9 0 9 0x8	I	addiu $9,$0,8	li $t1,8
0x00400004	9 0 10 0x6	I	addiu $10,$0,6	li $t2,6
0x00400008	9 0 11 0x0	I	addiu $11,$0,0	li $t3,0 # $t1*$t2
0x0040000c	0 0 10 1 - 42	R	slt $1,$0,$10	Loop: ble $t2,$0,Exit
0x00400010	4 1 0 0x3	I	beq $1,$0,3	
0x00400014	0 11 9 9 - 32	R	add $11,$11,$9	add $t3,$t3,$t1
0x00400018	8 10 10 0xffff	I	addi $10,$10,-1	addi $t2,$t2,-1
0x0040001c	2 100003	J	j 0x0040000c	j Loop
0x00400020	…	…	…	Exit: …

Summary

01 • **The MIPS ISA**: Registers, Instructions, Instruction Types

02 • **Binary Representation of Data**: Hexadecimal, Two’s Complement, Bitwise Operations

03 • **Binary Representation of ISA**: MIPS R-Format / I-Format / J-Format

Do These

- Read chapter 2 (2.13-2.19 optional)
- Do assignment 1

Next Time

- Introduction to pipelining
  - Building a datapath
  - Increasing performance using pipelining


centralbureaucracy.tumblr.com
What an awful dream! Ones and zeros everywhere! And I thought I saw a two.
It was just a dream, Bender. There's no such thing as two.